

Стандарт разметки HTML

Правила языка

Семантика

Используйте семантически корректные элементы. Для создания таблицы используйте [table](#), для списков – [ul](#) или [ol](#), для описания набора полей формы – [fieldset](#), для меток к полям – [label](#) и т.д.

Inline-стили крайне не приветствуются

Inline-стили усложняют код, нарушают принцип разделения разметки и оформления, увеличивают размер исходных кодов и затрудняют поддержку и редизайн страницы.

Стили нужно максимально выносить в CSS-файлы. Также нужно помнить о таких замечательных вещах как [colgroup](#) – этот тег позволяет применять некоторые стили (например ширину) сразу ко всем колонкам таблицы. Это избавляет не только от инлайновых стилей, но и от многократного навешивания классов на [td](#) и/или [tr](#).

Зачем вообще colgroup? Все же можно указать один раз в первой строке?

Для того, чтобы указывать это действительно один раз. Указав в `thead>colgroup` набор колонок и их ширины (например), вы можете работать с данными в `tbody` и никогда не задумываться, что надо выставлять какие-то ширины и т.п. Просто добавляйте или удаляйте данные.

Inline-JavaScript крайне не приветствуется

Все аналогично стилям. JavaScript-обработчики должны устанавливаться на элемент из внешних JS-файлов через CSS-селекторы.

Избегайте атрибутов, которые можно применить через CSS

Это ведет к затруднению разделения оформления и разметки, загромождает код.

```
1 <!-- Плохо -->
2 <td valign="top"></td>
3
4 <!-- Хорошо -->
5 <td class="topAligned"></td>

1 .topAligned {
2     vertical-align: top;
3 }
4 /* или */
5 td {
6     vertical-align: top;
7 }
```

В некоторых случаях объем написания будет такой же, в некоторых — даже чуть больше. Но это удобнее поддерживать. Для изменения стиливого оформления надо изменять CSS, а не HTML+CSS+PHP+что-то еще.

Всегда используйте корректный DOCTYPE

Это позволяет точно определять, в каком режиме браузеры будут отображать страницы и не допустить рендеринга в QuirksMode.

DOCTYPE указывается самой первой строкой HTML-документа, на одной строке.

Используемый при разработке DOCTYPE — XHTML Transitional:

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
```

"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">



Что такое QuirksMode?

Это режим "совместимости" в браузерах, при работе в котором браузер "будет исходить из предположения, что вы писали код страницы с ошибками и вольно отступали от стандартов, т.е. так, как писали в конце 90-ых годов". В частности в IE6 будет применяться блочная модель, по которой ширина объекта включает не только сам объект, но и его padding (в нормальной, стандартной блочной модели padding не входит в ширину объекта). Наличие корректного DOCTYPE – первый шаг к тому, что ваш HTML+CSS везде будет отображаться одинаково.

Размещайте CSS в начале страницы

Размещайте загрузку CSS в HEAD страницы. Это позволит браузеру быстрее начать рендеринг, пользователь быстрее увидит результат загрузки.

При этом недопустимо применение тегов [style](#) и размещение стилей в них. Стили должны размещаться в отдельных CSS-файлах. Это уменьшает вес страницы и повышает кэшируемость.

Размещайте JS в конце страницы

Загрузка JS, как правило, блокирует процесс рендеринга и подгрузки дополнительных ресурсов (картинки, CSS и т.п.). Размещение скриптов в конце страницы ускорит получение пользователем готовой страницы.

Предполагается, что все скрипты располагаются в отдельных файлах. В то же время допустимо использование кода внутри тега [script](#), если этого требует задача и/или особенности реализации.

Формирование URL для ссылок

Избегайте формирования абсолютных URL для внутренних страниц сайта. Допустимо использование абсолютных URL только для внешних ссылок, а также в тех случаях, когда без указания хоста не обойтись. Предпочтительны относительные URL.

Блочная верстка – крайне желательна

Блочная семантическая верстка способствует легкой смене дизайна страницы и более быстрому рендерингу страницы. Она проще читается, проще проводить ее рефакторинг.

Из минусов – блочная верстка более сложна для кроссбраузерной реализации.

Обработчики событий на ссылках

При установке обработчика на ссылки чаще всего требуется, чтобы сама ссылка не отработывала. Для этого применяются два метода: либо в обработчике возвращаем false (при этом href="#"), либо в обработчике не происходит ничего дополнительного, но href="javascript:void(0)". Нужно быть внимательным с первым вариантом: если обработчик не будет возвращать false, то ссылка сработает, и браузер отскроллирует страницу на самый верх.

В то же время второй вариант не является предпочтительным, т.к. используя то или иное возвращаемое значение можно управлять дальнейшим всплытием события.

Валидация страниц

Все страницы должны проходить валидацию (см. <http://validator.w3.org>) – дополнительный способ обезопасить себя от различных кроссбраузерных ошибок, и, банально, от незакрытых тегов.

Страницы могут иметь ошибки валидации, но, как правило, наличие ошибок должно быть обусловлено чем-либо (кроссбраузерные хаки, т.п.).

Кодировка файлов

Используйте одну и ту же кодировку для всех компонентов страницы (HTML, CSS, JS). Это избавит вас от лишних проблем с отображением страницы и динамического контента, генерируемого в JS (имеются ввиду, естественно, проблемы с кодировкой).

Стиль оформления кода

- Табуляция в три пробела.
- теги: все буквы маленькие.
- Форматирование атрибутов. После названия атрибута нет пробела перед =, после = тоже нет пробела. Атрибуты всегда в кавычках.
- Названия идентификаторов объектов на английском языке в стиле **lowerCamelCase** (нижний регистр первой буквы). Идентификатор должен иметь осмысленное значение. Например, id1 – плохое значение идентификатора, но допустимо при автоматической генерации объектов.